

Conception d'un Réseau de Neurones Fully Connected

Application pour la classification de la base de données MNIST

L'objectif de ce projet est de concevoir un réseau de neurones en C sans utiliser de bibliothèques prévues à cet effet (vous pourrez utiliser les bibliothèques standards du C). Pour simplifier, nous nous intéresserons uniquement aux réseaux dits *fully connected*, c'est-à-dire des réseaux dans lesquels les entrées d'un neurone sont connectées à l'ensemble des sorties de la couche précédente. Parmi les bases de données couramment utilisées, on trouve la base de données MNIST, constituée d'un ensemble d'images de chiffres manuscrits de taille 28×28 en niveaux de gris. Il vous faudra créer et entraîner votre réseau afin qu'il soit capable de reconnaître le chiffre représenté sur une image.

1 Base de données

La base de données MNIST est couramment utilisée pour l'évaluation de modèles de classification d'images. Elle est constituée d'images de chiffres manuscrits allant de 0 à 9, chaque image étant de taille 28×28 pixels en niveaux de gris.

Une base de données de classification est un ensemble de données (ici des images) associées à des étiquettes (ici 10 classes indexées de 0 à 9). Chaque donnée et son étiquette forment un couple, l'étiquette correspondant à la vraie classe de la donnée. Si le réseau fonctionne correctement, la classe estimée à partir de la donnée doit correspondre à celle indiquée par l'étiquette. Par la suite, on notera ce couple $\{x_i, y_i\}$, correspondant au i -ème élément du dataset. Ici, x_i est un vecteur dont chaque composante correspond à un pixel de l'image, et y_i est le chiffre représenté.

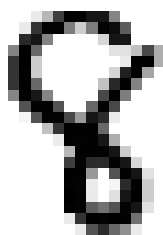


FIGURE 1 – Chiffre 8 manuscrit issu de la base de données MNIST

Dans le cadre de l'apprentissage automatique, on ne travaille généralement pas sur l'ensemble des données en une seule fois, mais on distingue deux sous-ensembles appelés *datasets*. Le dataset d'entraînement (*training set*) contient les données utilisées pour apprendre les paramètres du modèle, c'est-à-dire les poids et les biais du réseau de neurones.

Une fois l'entraînement terminé, on utilise un dataset de test (*test set*), qui contient des données que le réseau n'a jamais vues durant l'apprentissage. Ce jeu de données permet d'évaluer la capacité du réseau à généraliser, c'est-à-dire à produire des prédictions correctes sur de nouvelles données. La qualité de la classification sur le dataset de test permet ainsi d'évaluer les performances du modèle entraîné.

Reconnaître un symbole manuscrit est un problème complexe en informatique, au point qu'il en découle toute un champ de recherche : la ROC (Reconnaissance Optique de Caractères). Nous n'avons pas attendu les réseaux de neurones pour réussir à résoudre ce type de problème, mais ils se révèlent particulièrement efficaces pour cette tâche. C'est donc cette approche que nous allons utiliser dans ce projet.

2 Réseau de neurones

Un réseau de neurones est un modèle informatique dont le fonctionnement est inspiré de celui des neurones biologiques. Il est composé de couches correspondant à des ensembles de neurones n'étant pas connectés entre eux, mais reliés aux couches précédente et suivante. La première couche reçoit les données d'entrée sous forme de vecteur, la dernière fournit le résultat sous forme d'un vecteur de sortie, et les couches intermédiaires, dites *hidden layers*, peuvent varier en nombre et en taille.

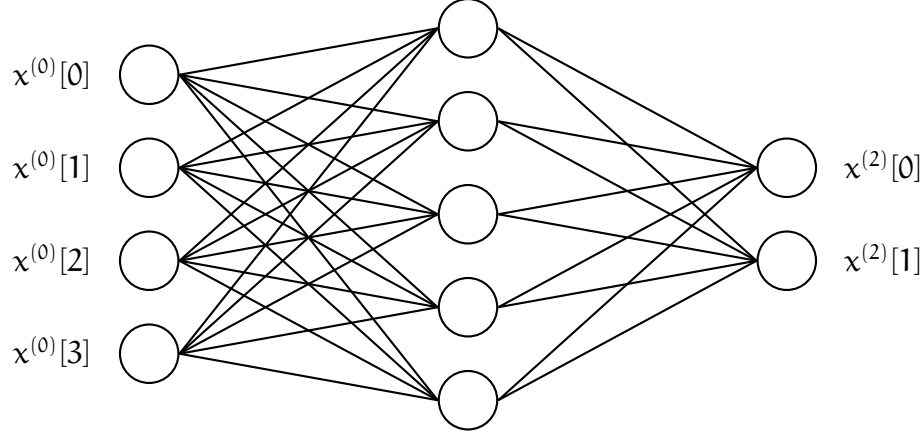


FIGURE 2 – Réseau de neurones entièrement connecté avec une couche en entrée de 4 neurones, un *hidden layer* de 5 neurones et une couche de sortie de 2 neurones

De manière générale, un réseau de neurones reçoit en entrée un vecteur $x_i^{(0)} \in \mathbb{R}^{N_0}$. À chaque couche, une transformation est appliquée à ce vecteur. Cette transformation est composée d'une application linéaire, suivie de l'application d'une fonction non linéaire, indépendamment sur chaque composante du vecteur. Après cette transformation, le vecteur peut changer de dimension et devenir $x_i^{(1)} \in \mathbb{R}^{N_1}$. Ce procédé est répété pour chaque couche jusqu'à la dernière, la $K^{\text{ème}}$ couche, qui fournit le vecteur de sortie du réseau $x_i^{(K)} \in \mathbb{R}^{N_K}$.

Dans notre cas, nous souhaitons réaliser une tâche de classification sur la base de données MNIST, c'est-à-dire déterminer à quelle classe appartient une image, ce qui revient ici à identifier le chiffre qui y est représenté. Il nous faut donc 10 classes, ce qui correspond à la taille du vecteur de sortie du réseau. Pour déterminer la classe associée au vecteur $x_i^{(0)} \in \mathbb{R}^{N_0}$ estimée par le réseau, l'une des méthodes consiste à choisir l'argmax du vecteur de sortie. Ainsi, en notant $\hat{y}_i$ la classe estimée par le réseau pour l'image x_i , on écrit

$$\hat{y}_i = \operatorname{argmax} x_i^{(K)} \quad (1)$$

Autrement dit, la classe estimée correspond à l'indice de la composante de valeur maximale dans le vecteur de sortie du réseau.

L'intérêt principal des réseaux de neurones réside dans le fait que, mis à part l'enchaînement des couches, nous n'indiquons pas explicitement au réseau comment calculer le vecteur $x_i^{(K)}$. Chaque neurone contient des paramètres (poids et des biais) que le réseau doit apprendre, c'est-à-dire déterminer automatiquement les valeurs des paramètres permettant de réaliser au mieux la tâche de classification. On définit ainsi une structure de réseau, et celui-ci apprend les poids et biais adaptés à partir des données.

3 Inférence

L'inférence d'un réseau de neurones correspond à la phase durant laquelle le modèle utilise les paramètres appris (poids et biais) pour produire une prédiction à partir de nouvelles données d'entrée. Contrairement à la phase d'apprentissage, aucune mise à jour des paramètres n'est effectuée, le réseau se contente d'appliquer successivement les transformations définies par chaque couche afin de calculer la sortie associée à une entrée donnée. Dans le cadre de la classification, l'inférence permet ainsi d'estimer la classe la plus probable correspondant à la donnée fournie.

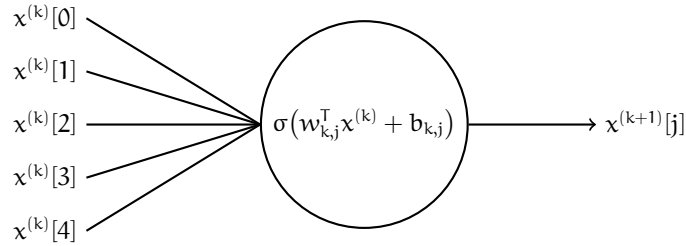


FIGURE 3 – Schéma du neurone j de la couche k

Afin de formaliser le fonctionnement d'un réseau de neurones, nous commençons par en donner une description mathématique à l'échelle d'un neurone. Cette description élémentaire permettra ensuite de généraliser naturellement au cas d'une couche, puis à celui du réseau complet.

L'équation associée à un neurone s'écrit

$$x^{(k+1)}[j] = \sigma(w_{k,j}^T x^{(k)} + b_{k,j}), \quad (2)$$

avec :

- $x^{(k+1)} \in \mathbb{R}^{N_{k+1}}$ le vecteur de sortie de la couche k
- $x^{(k)} \in \mathbb{R}^{N_k}$ le vecteur d'entrée de la couche k
- $w_{k,j} \in \mathbb{R}^{N_k}$ le vecteur des poids du neurone j
- $b_{k,j} \in \mathbb{R}$ le biais du neurone j
- $\sigma(\cdot) : \mathbb{R} \mapsto \mathbb{R}$ la fonction d'activation non linéaire

On travaille généralement sur l'équation de la couche entière, que l'on écrit :

$$x^{(k+1)} = \sigma(W_k x^{(k)} + B_k), \quad (3)$$

avec :

- $W_k \in \mathbb{R}^{N_k \times N_{k+1}}$ la matrice des poids de la couche k
- $B_k \in \mathbb{R}^{N_{k+1}}$ le vecteur des biais de la couche k
- $\sigma(\cdot) : \mathbb{R} \mapsto \mathbb{R}$ la fonction d'activation appliquée composante par composante

Parmi les fonctions d'activation couramment utilisées, on trouve par exemple la fonction ReLU et la fonction sigmoïde :

$$\text{ReLU}(x) = \max(x, 0), \quad \text{sigmoid}(x) = \frac{1}{1 + e^{-x}}. \quad (4)$$

On peut ainsi représenter le réseau de la façon suivante :

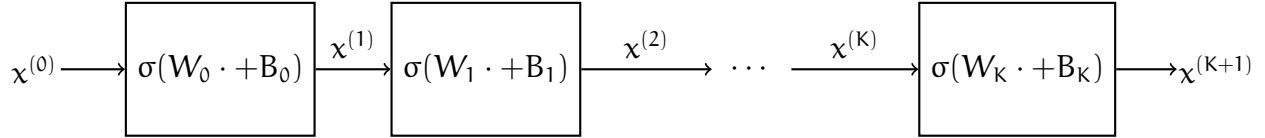


FIGURE 4 – Schéma bloc d'un réseau de neurone à K couches

Si l'on prend l'exemple d'un réseau de trois couches, on peut écrire l'estimation réalisée lors de l'étape d'inférence de la façon suivante :

$$y_i = \text{argmax} \left[\sigma(W_2 \sigma(W_1 \sigma(W_0 x_i + B_0) + B_1) + B_2) \right] \quad (5)$$

4 Apprentissage

L'entraînement d'un réseau de neurones consiste à ajuster automatiquement ses paramètres, à savoir les poids et les biais, afin de minimiser un critère mesurant l'écart entre les prédictions du réseau (classes estimées) et les sorties attendues (labels). Cette phase repose sur l'utilisation d'un ensemble de données d'apprentissage et sur l'optimisation itérative des paramètres à l'aide d'algorithmes de descente de gradient. À chaque itération, le réseau évalue ses prédictions, calcule l'erreur associée, puis met à jour ses paramètres de manière à améliorer progressivement ses performances.

Dans la partie suivante, afin de simplifier les calculs, nous allons considérer un réseau de neurones sans biais B_i . En pratique on peut même pour un réseau avec des biais considérer que les seuls paramètres à apprendre sont des poids :

$$Wx + B = \left[W \mid B \right] \begin{bmatrix} x \\ 1 \end{bmatrix} = \tilde{W}\tilde{x}.$$

Pour entraîner le réseau, il est nécessaire de définir ce critère. Dans le contexte des réseaux de neurones, ce critère est appelé la fonction de perte, ou *loss*, notée ℓ . Elle permet de comparer la sortie du réseau $x_i^{(K)}$ à la sortie désirée. Par souci de simplicité dans ce projet, nous utiliserons la fonction *one-hot* δ , définie par

$$\delta : \mathbb{N} \mapsto \{0, 1\}^{10} \quad (6)$$

$$\delta(y)_i = \begin{cases} 1 & \text{si } i = y \\ 0 & \text{sinon} \end{cases} \quad (7)$$

On définit alors la fonction de perte par

$$\ell : \mathbb{R}^{10} \times \mathbb{N} \mapsto \mathbb{R} \quad (8)$$

$$\ell(x, y) = \|x - \delta(y)\|_2^2 \quad (9)$$

Pendant l'apprentissage, on cherche à minimiser cette *loss*. En effet, si pour chaque échantillon on a $x_i = \delta(y_i)$, c'est-à-dire si le réseau réalise une classification parfaite sans erreur, alors la perte est nulle. On cherche donc à minimiser cette perte sur l'ensemble des échantillons du dataset d'apprentissage, ce qui revient à minimiser l'espérance de la *loss* sur le *train set* $\mathcal{D}$:

$$\mathbb{E}_{(x,y) \sim \mathcal{D}}[\ell(x, y)] \quad (10)$$

Pour cela, on utilise l'algorithme de descente de gradient stochastique (*Stochastic Gradient Descent*, SGD) :

Algorithm 1 Stochastic Gradient Descent pour un réseau de neurones

Initialiser les poids W_k avec des entrées tirées selon $\mathcal{N}(0, 1)$ pour $k \in \llbracket 0, K - 1 \rrbracket$

for $i \in \llbracket 0, N - 1 \rrbracket$ **do**

 Calculer les gradients $\frac{d\ell}{dW_k}$ par rétropropagation

for $k \in \llbracket 0, K - 1 \rrbracket$ **do**

$W_k \leftarrow W_k - \eta \frac{d\ell}{dW_k}$

end for

end for

Comme on le voit dans l'algorithme ci-dessus, l'apprentissage du réseau fait intervenir le calcul de la dérivée de la *loss*, que l'on peut expliciter de la façon suivante :

$$\begin{aligned} \frac{d\ell}{dW_i} &:= \frac{d\ell(x^{(K)}, y)}{dW_i} \\ &= \frac{\partial \ell}{\partial x^{(K)}} \frac{dx^{(K)}}{dW_i} \\ &= \frac{\partial \ell}{\partial x^{(K)}} \frac{dx^{(K)}}{dx^{(K-1)}} \frac{dx^{(K-1)}}{dW_i} \\ &= \frac{\partial \ell}{\partial x^{(K)}} \frac{dx^{(K)}}{dx^{(K-1)}} \frac{dx^{(K-1)}}{dx^{(K-2)}} \cdots \frac{dx^{(i+1)}}{dW_i}. \end{aligned}$$

On remarque ici que le calcul de la *loss* fait intervenir le terme $\frac{\partial \ell}{\partial \mathbf{x}^{(K)}} \prod_{j=K}^i \frac{d\mathbf{x}^{(j)}}{d\mathbf{x}^{(j-1)}}$. Ainsi, lors du calcul de $\frac{d\ell}{d\mathbf{W}_{i-1}}$, nous pourrions réutiliser $\frac{\partial \ell}{\partial \mathbf{x}^{(K)}} \prod_{j=K}^i \frac{d\mathbf{x}^{(j)}}{d\mathbf{x}^{(j-1)}}$. Il est donc plus intéressant, d'un point de vue algorithmique, de commencer par calculer la dérivée par rapport aux poids de la dernière couche et de remonter jusqu'à la première. C'est ce que l'on appelle la *backpropagation*.

L'expression de la dérivée d'un layer par rapport au précédent est assez simple : $\frac{d\mathbf{x}^{(j)}}{d\mathbf{x}^{(j-1)}} = \mathbf{W}_{j-1}$, mais celle d'un layer par rapport aux poids est un peu plus délicate.

$$\begin{aligned} \frac{d\mathbf{x}^{(i+1)}}{d\mathbf{W}_i} &= \frac{d}{d\mathbf{W}_i} \sigma(\mathbf{W}_i \mathbf{x}^{(i)}) \\ &= \dot{\sigma}(\mathbf{W}_i \mathbf{x}^{(i)}) \frac{\partial (\mathbf{W}_i \mathbf{x}^{(i)})}{\partial \mathbf{W}_i}. \end{aligned}$$

Cette dérivée d'un vecteur par rapport à notre matrice de poids n'est pas une matrice, mais un objet tridimensionnel (dépendant de trois indices). Afin de mieux comprendre sa structure, nous allons examiner chaque entrée de cet objet. Pour éviter toute confusion, nous omettrons dans les notations suivantes l'indice i lié à la couche, afin de ne pas le confondre avec ceux associés aux dimensions de l'objet.

$$\frac{\partial (\mathbf{W}\mathbf{x})}{\partial \text{vec}(\mathbf{W})} = \left\{ \frac{\partial [\mathbf{W}\mathbf{x}]_k}{\partial \mathbf{W}_{n,m}} \right\}_{k,n,m}, \text{ et donc } \frac{\partial [\mathbf{W}\mathbf{x}]_k}{\partial \mathbf{W}_{n,m}} = \begin{cases} x_m & \text{si } n = k \\ 0 & \text{sinon} \end{cases} \quad (11)$$

Pour visualiser cet objet on peut considérer que la matrice des poids $\mathbf{W}$ peut s'écrire comme un vecteur en alignant ses lignes. A partir de l'équation (11) on obtient alors que

$$\frac{\partial (\mathbf{W}\mathbf{x})}{\partial \mathbf{W}} = \left[\begin{array}{c|c|c|c} \begin{smallmatrix} n=0 \\ m \in \llbracket 0, M-1 \rrbracket \end{smallmatrix} & \begin{smallmatrix} n=1 \\ m \in \llbracket 0, M-1 \rrbracket \end{smallmatrix} & \begin{smallmatrix} n=2 \\ m \in \llbracket 0, M-1 \rrbracket \end{smallmatrix} & \begin{smallmatrix} n=3 \\ m \in \llbracket 0, M-1 \rrbracket \end{smallmatrix} \\ \hline \begin{smallmatrix} x^T \\ 0 \\ 0 \\ 0 \end{smallmatrix} & \begin{smallmatrix} 0 \\ x^T \\ 0 \\ 0 \end{smallmatrix} & \begin{smallmatrix} 0 \\ 0 \\ x^T \\ 0 \end{smallmatrix} & \begin{smallmatrix} 0 \\ 0 \\ 0 \\ x^T \end{smallmatrix} \end{array} \right]_{k \in \llbracket 0, K-1 \rrbracket} \quad (12)$$

Et donc on en déduit que

$$\dot{\sigma}(\mathbf{W}_i \mathbf{x}^{(i)}) \frac{\partial \mathbf{W}_i \mathbf{x}^{(i)}}{\partial \mathbf{W}_i} = \dot{\sigma}(\mathbf{W}_i \mathbf{x}^{(i)}) \mathbf{x}^{(i)T}.$$

5 Travail demandé

Ce projet est relativement avancé, tant sur le plan théorique que sur le plan technique de la programmation. Il n'est donc pas nécessairement attendu que vous validiez tous les points ci-dessous. Mais de nombreuses améliorations sont possibles ! Néanmoins le travail demandé est le suivant :

1. Comprendre comment calculer $\frac{d\ell}{dW_i}$ et $\frac{d\ell}{dB_i}$.
2. Réaliser une fonction permettant de créer un réseau avec un nombre variable de couches de tailles variables.
3. Réaliser une fonction permettant l'inférence du réseau.
4. Réaliser une fonction permettant de mesurer la qualité des inférences du réseau.
5. Réaliser une fonction permettant l'apprentissage du réseau.